Project 2.5 Report

Phase Detector

Anthony Betancourt
David Shaowen Wang
WES 237C: Oct 22, 2026

Overview

The phase detector is a complex matched filter followed by a CORDIC in vectoring mode. The matched filter is built from four real 32-tap FIR filters (firI1, firI2, firQ1, firQ2) with coefficients of +1 or -1, reusing the Project 1 FIR structure. The filter outputs are combined as X = FIR_I1(I) + FIR_Q2(Q) and Y = FIR_I2(Q) - FIR_Q1(I). The CORDIC then converts (X, Y) to polar form (R, Theta) using 16 iterations in single-precision floating point, with the final x value scaled by 1/K = 0.607252935 and a +/-pi correction when X is negative.

The top-level function follows the project specification and processes one I/Q sample per call: void phasedetector(data_t I, data_t Q, data_t *R, data_t *Theta). The 1024-sample loop is in the testbench. The function calls fir() and then cordiccart2pol(). At a high level, the project required us to:

- Build and verify the matched filter on its own (`fir_top`) and report its baseline synthesis results
- Build and verify the complete phase detector from the FIR and CORDIC blocks and report its baseline results
- Optimize the phase detector with pragmas into at least two versions and compare resources and throughput (`phasedetector_optimized1`, `phasedetector_optimized2`)
- Answer Question 1: the throughput of the phase detector and how it relates to its FIR and CORDIC components
- Run the design on the Pynq-Z2 with AXI-Stream interfaces and two DMAs (PYNQ Demo)

Link to Github:

The code for the design in this report is available at the following URL:

1. Github Repo link: https://github.com/UCSD-237C-2026/wes-237c-project-2-cordic-group-5.git

Design Environment

Table 1: Tools and settings used for every design

Item	Setting
HLS tool	Vitis HLS 2024.2 (Vivado 2024.2 for the block design)
Target part	xc7z020clg400-1 (Pynq-Z2), 220 DSPs, 106,400 FFs, 53,200 LUTs
Target clock	10 ns (100 MHz)

Top function	phasedetector(data_t I, data_t Q, data_t *R, data_t *Theta) in phasedetector.cpp; fir_top uses fir(...) in fir.cpp
Data type	data_t = float (single precision) for every variable
Accuracy check	RMSE of R and Θ against the golden file out.gold.dat, reported by phasedetector_test.cpp (pass if below 1e-3)
C synthesis	Makefile flow: make report runs C synthesis and the performance and resource table is copied to the .rpt file in each design folder

Functional verification

Table 1. C-simulation results (1024 samples, error threshold 1e-3).

Design	RMSE (R)	RMSE (Theta)	Result
fir_top (matched filter only)	3.918e-7	-	Pass
phasedetector (FIR + CORDIC)	3.857e-5	3.163e-7	Pass
phasedetector, streaming wrapper (AXI-Stream, TLAST checked)	3.857e-5	3.163e-7	Pass, TLAST only on last output

The R error is dominated by small offsets in the golden file (for example, a golden R of 2.000005 for an exact value of 2). The optimized versions only add pragmas, so their C behavior is identical.

Baseline results

Table 2. Baseline synthesis results (no optimization pragmas).

Design	Latency (cycles)	Interval (cycles)	Est. clock (ns)	BRAM	DSP	FF	LUT
fir_top	210	211	7.256	8 (2%)	20 (9%)	2652 (2%)	3783 (7%)
phasedetector	383	384	8.424	8 (2%)	20 (9%)	3150 (2%)	4613 (8%)

The baseline phase detector accepts one sample every 384 cycles. Vitis HLS pipelined each inner loop automatically but could not reach an initiation interval (II) of 1. The FIR accumulation loops reach II = 6 because each addition depends on the previous floating-point add (5-cycle latency). The CORDIC loop reaches II = 10 because each iteration depends on the previous one through a floating-point multiply, add and select. The four FIR filters run in parallel and take about 199 cycles (about 52% of the total), then the CORDIC takes about 170 cycles (about 45%). The two stages run back to back and nothing overlaps between samples .

Optimization results

Table 3. Phase detector optimization rounds (scalar, one sample per call).

Version	Pragmas	Latency (cycles)	Interval (cycles/sample)	Est. clock (ns)	DSP	FF	LUT
Baseline	none	383	384	8.424	20 (9%)	3150 (2%)	4613 (8%)
optimized1	PIPELINE II=4 on the function; ARRAY_PARTITION complete on the four shift registers	312	4	8.567	116 (52%)	36103 (33%)	30243 (56%)
optimized2	same as optimized1 with PIPELINE II=3	316	3	8.495	150 (68%)	42405 (39%)	38037 (71%)

Pipelining the whole function makes Vitis HLS fully unroll the FIR loops (31 iterations each) and the 16-iteration CORDIC loop. The loop-carried dependencies disappear, and the floating-point operators are shared across II cycles. With II = 4 (optimized1) the design accepts one sample every 4 cycles, 96 times the baseline throughput, using 116 DSPs (52%), 56% of the LUTs and no BRAM, because the shift registers become flip-flops. With II = 3 (optimized2) throughput rises another 33% (128 times the baseline) at the cost of 34 more DSPs (+29%) and about 7,800 more LUTs (+26%). Resource use scales roughly as 1/II, so the extra throughput is not free. An II of 2 would need roughly 230 DSPs and would not fit on the xc7z020 (220 DSPs).

Timing note: the estimated clock period grew from 8.4 ns to about 8.5 ns for the shared designs. With the 2.70 ns uncertainty margin, the estimate exceeds the 10 ns target, so meeting 100 MHz after place and route is not guaranteed and must be confirmed in Vivado .

Question 1: throughput of the phase detector

The baseline phase detector processes one I/Q sample every 384 cycles (about 3.84 us at 100 MHz, or roughly 260 thousand samples per second). Throughput is limited by the sum of its two stages. The FIR matched filter takes about 199 cycles and the CORDIC about 170 cycles, and they execute one after the other with no overlap between consecutive samples. Both stages are limited by loop-carried dependencies through floating-point operators: the FIR accumulation (II = 6) and the CORDIC iteration (II = 10).

Table 5: Throughput of the phase detector designs at 100 MHz (steady state, excluding the pipeline fill latency)

Design	Interval (cycles)	Throughput (MS/s)	Time for 1024 samples	Speed-up
fir_top baseline	211	0.474	2.16 ms	n/a
Phase detector baseline	384	0.260	3.93 ms (393,216 cycles)	1×
optimized1 (II = 4)	4	25.0	41.0 µs (4,096 cycles)	96×
optimized2 (II = 3)	3	33.3	30.7 µs (3,072 cycles)	128×

Pipelining the entire phase detector function with an initiation interval of 4 (optimized1) lets a new sample enter every 4 cycles. This is 25 million samples per second at 100 MHz, a 96x improvement, and 1024 samples take about 4,100 cycles (41 us) instead of about 393,000 cycles (3.9 ms). The price is area: DSPs rise from 20 to 116 and LUTs from 4,613 to 30,243.

Lowering the II to 3 (optimized2) gives 128x the baseline throughput but uses 150 DSPs and 71% of the LUTs, which leaves less room for the DMA and interconnect logic in a full design.

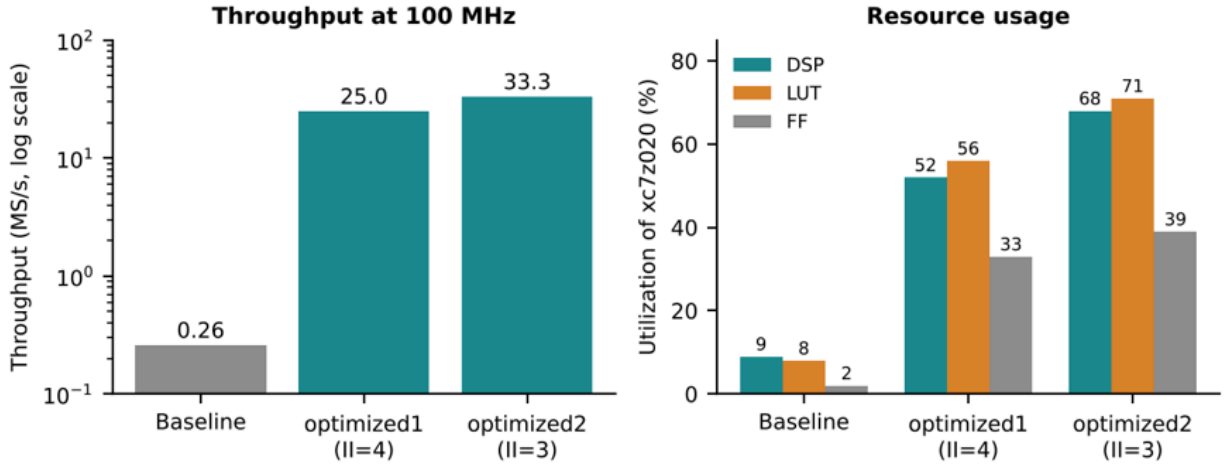


Figure 5: Throughput (left) and resource usage (right) of the baseline and the two optimized designs

Further improvements are: (1) restructure the FIR accumulation as a tree of adders or partial sums so the add chain no longer limits the II; (2) overlap the FIR and CORDIC stages with a dataflow or pipelined structure so the two stages work on different samples; (3) reduce the number of CORDIC iterations if the accuracy allows; and (4) replace single-precision floating point with fixed point, which removes the long-latency floating-point operators and cuts DSP and LUT use substantially.

PYNQ Demo

The course block-design script (overlay.tcl) was used to generate the Vivado design: Zynq PS, two AXI DMAs (I/R on axi_dma_0 and Q/Theta on axi_dma_1) sharing HP0 through an AXI interconnect, and the phase detector IP. The host notebook was completed to load the overlay, write the length register, start the IP, run both DMAs and plot R and Theta against the golden data. The bitstream build was started but did not finish before the deadline, so the design was not run on the Pynq-Z2 and there is no board output to show. The Demo folder therefore contains the completed host.ipynb but no .bit or .hwh file. The notebook is untested on hardware.